



Singularry USDD Vault

AUDIT REPORT

Version 1.0.0

Serial No. 2026011600012021

Presented by Fairyproof

January 16, 2026

01. Introduction

This document includes the results of the audit performed by the Fairyproof team on the Singularry USDD Vault project.

Audit Start Time:

January 13, 2026

Audit End Time:

January 15, 2026

Audited Source File's Address:

<https://github.com/singularry/sly-usd-vault/tree/ba7692b17e912080f2a54fc47903f927a3ce08d2>

<https://github.com/singularry/slywallet-contracts/tree/feature/usdd-staking-facet>

Audited Code's Github Repository:

<https://github.com/singularry/slywallet-contracts/tree/feature/usdd-staking-facet>

Audited Code's Github Commit Number When Audit Started:

400f09c677dc4b97bfe891cda259d26d74d3fd5b

Audited Code's Github Commit Number When Audit Ended:

5c5dce9a989b2df6dd580b865373dcfa3c0921cd

Audited Source Files:

The source files audited include all the files as follows:

```
usdd
├── ISLYWalletUSDDVaultFacet.sol
├── SLYWalletUSDDVaultFacet.sol
└── SLYWalletUSDDVaultStorage.sol
```

1 directory, 3 files

```
contracts
├── USDDVault.sol
├── interfaces
│   ├── IPSM.sol
│   ├── ISLYFeeManagement.sol
│   └── IUSDDVault.sol
```

2 directories, 4 files

The goal of this audit is to review Singularity's solidity implementation for its USDD Vault function, study potential security vulnerabilities, its general design and architecture, and uncover bugs that could compromise the software in production.

We make observations on specific areas of the code that present concrete problems, as well as general observations that traverse the entire codebase horizontally, which could improve its quality as a whole.

This audit only applies to the specified code, software or any materials supplied by the Singularity team for specified versions. Whenever the code, software, materials, settings, environment etc is changed, the comments of this audit will no longer apply.

— Disclaimer

Note that as of the date of publishing, the contents of this report reflect the current understanding of known security patterns and state of the art regarding system security. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. If the audited source files are smart contract files, risks or issues introduced by using data feeds from offchain sources are not extended by this review either.

Given the size of the project, the findings detailed here are not to be considered exhaustive, and further testing and audit is recommended after the issues covered are fixed.

To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

— Methodology

The above files' code was studied in detail in order to acquire a clear impression of how the its specifications were implemented. The codebase was then subject to deep analysis and scrutiny, resulting in a series of observations. The problems and their potential solutions are discussed in this document and, whenever possible, we identify common sources for such problems and comment on them as well.

The Fairproof auditing process follows a routine series of steps:

1. Code Review, Including:
 - Project Diagnosis

Understanding the size, scope and functionality of your project's source code based on the specifications, sources, and instructions provided to Fairproof.

- Manual Code Review

Reading your source code line-by-line to identify potential vulnerabilities.

- Specification Comparison

Determining whether your project's code successfully and efficiently accomplishes or executes its functions according to the specifications, sources, and instructions provided to Fairproof.

2. Testing and Automated Analysis, Including:

- Test Coverage Analysis

Determining whether the test cases cover your code and how much of your code is exercised or executed when test cases are run.

- Symbolic Execution

Analyzing a program to determine the specific input that causes different parts of a program to execute its functions.

3. Best Practices Review

Reviewing the source code to improve maintainability, security, and control based on the latest established industry and academic practices, recommendations, and research.

— Structure of the document

This report contains a list of issues and comments on all the above source files. Each issue is assigned a severity level based on the potential impact of the issue and recommendations to fix it, if applicable. For ease of navigation, an index by topic and another by severity are both provided at the beginning of the report.

— Documentation

For this audit, we used the following source(s) of truth about how the token issuance function should work:

Website: <https://singularry.org/>

Source Code:

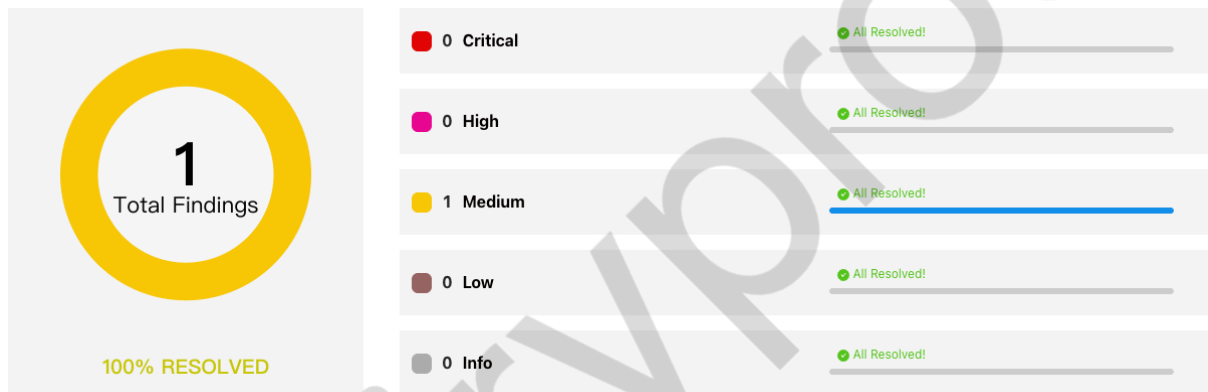
<https://github.com/singularry/sly-usd-vault/tree/ba7692b17e912080f2a54fc47903f927a3ce08d2>

<https://github.com/singularry/slywallet-contracts/tree/feature/usdd-staking-facet>

These were considered the specification, and when discrepancies arose with the actual code behavior, we consulted with the Singularry team or reported an issue.

— Comments from Auditor

Serial Number	Auditor	Audit Time	Result
2026011600012021	Fairyproof Security Team	Jan 13, 2026 - Jan 15, 2026	Passed



Summary:

The Fairyproof security team used its auto analysis tools and manual work to audit the project. During the audit, one issue of medium-severity was uncovered. The Singularry team fixed one issue of medium, and acknowledged the remaining issues.

02. About Fairyproof

[Fairyproof](#) is a leading technology firm in the blockchain industry, providing consulting and security audits for organizations. Fairyproof has developed industry security standards for designing and deploying blockchain applications.

03. Introduction to Singularity

Singularity is a platform of creating next-generation financial systems with intelligent, autonomous protocols and seamless cross-chain operations.

The above description is quoted from relevant documents of Singularity.

04. Major functions of audited code

The audited module implements a USDD staking Vault. Users deposit USDT, which the Vault converts to USDD and then stakes as sUSDD to earn yield. Depositing USDT requires locking a configured minimum amount of SingularityToken in the Vault as collateral. SlyWallet integrates native Vault interactions, allowing users to perform deposits, conversions, staking, and withdrawals directly through the SlyWallet interface.

05. Coverage of issues

The issues that the Fairyproof team covered when conducting the audit include but are not limited to the following ones:

- Access Control
- Admin Rights
- Arithmetic Precision
- Code Improvement
- Contract Upgrade/Migration
- Delete Trap
- Design Vulnerability
- DoS Attack
- EOA Call Trap
- Fake Deposit
- Function Visibility
- Gas Consumption
- Implementation Vulnerability
- Inappropriate Callback Function
- Injection Attack
- Integer Overflow/Underflow

- IsContract Trap
- Miner's Advantage
- Misc
- Price Manipulation
- Proxy selector clashing
- Pseudo Random Number
- Re-entrancy Attack
- Replay Attack
- Rollback Attack
- Shadow Variable
- Slot Conflict
- Token Issuance
- Tx.origin Authentication
- Uninitialized Storage Pointer

06. Severity level reference

Every issue in this report was assigned a severity level from the following:

Critical severity issues need to be fixed as soon as possible.

High severity issues will probably bring problems and should be fixed.

Medium severity issues could potentially bring problems and should eventually be fixed.

Low severity issues are minor details and warnings that can remain unfixed but would be better fixed at some point in the future.

Informational is not an issue or risk but a suggestion for code improvement.

07. Major areas that need attention

Based on the provided source code the Fairyproof team focused on the possible issues and risks related to the following functions or areas.

- Function Implementation

We checked whether or not the functions were correctly implemented.
We didn't find issues or risks in these functions or areas at the time of writing.

- Access Control

We checked each of the functions that could modify a state, especially those functions that could only be accessed by owner or administrator
We didn't find issues or risks in these functions or areas at the time of writing.

- Token Issuance & Transfer

We examined token issuance and transfers for situations that could harm the interests of holders.
We didn't find issues or risks in these functions or areas at the time of writing.

- State Update

We checked some key state variables which should only be set at initialization.
We didn't find issues or risks in these functions or areas at the time of writing.

- Asset Security

We checked whether or not all the functions that transfer assets were safely handled.
We found one issue, for more details please refer to [FP-1] in "09. Issue description".

- Miscellaneous

We checked the code for optimization and robustness.
We didn't find issues or risks in these functions or areas at the time of writing.

08. List of issues by severity

Index	Title	Issue/Risk	Severity	Status
FP-1	Missing Emergency Withdraw Function: Users Unable to Retrieve Funds When Vault Is Paused	Design Vulnerability	Medium	✓ Fixed

09. Issue descriptions

[FP-1] Missing Emergency Withdraw Function: Users Unable to Retrieve Funds When Vault Is Paused

Design Vulnerability

Medium

✓ Fixed

Issue/Risk: Design Vulnerability

Description:

The USDDVault contract did not originally provide an emergency withdrawal mechanism. In extreme scenarios—such as when the contract is paused or the main withdrawal logic fails—users would be unable to retrieve their staked assets, resulting in a risk of funds being permanently locked. While the contract supports a pause mechanism, it lacked a user-accessible emergency exit, which is considered a best practice for DeFi protocols to ensure user fund safety and availability.

Recommendation:

It is recommended to add an `emergencyWithdrawShares` function that allows users to withdraw all their sUSDD shares directly when the contract is paused, bypassing the normal redemption flow. This function should follow the Checks-Effects-Interactions (CEI) pattern, update user and global state before transferring tokens, and only be callable when the contract is paused.

Update:

The project team has implemented the recommendation by adding the `emergencyWithdrawShares` function. This function enables users to withdraw all their sUSDD shares directly when the contract is paused, significantly improving user fund safety in emergency situations.

Status:

The issue is fixed at commit of `5c5dce9a989b2df6dd580b865373dcfa3c0921cd`.

10. Recommendations to enhance the overall security

We list some recommendations in this section. They are not mandatory but will enhance the overall security of the system if they are adopted.

- Consider managing the owner's access control with great care and transferring it to a multi-sig wallet or DAO when necessary.

11. Appendices

11.1 Unit Test

1. USDDVault.t.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "forge-std/Test.sol";
import "../src/sly_usdd_vault/USDDVault.sol";
import "../src/sly_usdd_vault/interfaces/IUSDDVault.sol";
import "../src/common/mocks/MockERC20.sol";
import "../src/common/mocks/MockPSM.sol";
import "../src/common/mocks/MockSLYFeeManagement.sol";
import "../src/common/mocks/MockSUSDD.sol";
import {Pausable} from "@openzeppelin/contracts/utils/Pausable.sol";

contract USDDVaultTest is Test {
    USDDVault public vault;
    MockERC20 public singularrayToken;
    MockERC20 public usdt;
    MockERC20 public usdd;
    MockSUSDD public sUsdd;
    MockPSM public psm;
    MockSLYFeeManagement public feeManager;

    address public owner = address(1);
    address public feeReceiver = address(2);
    address public alice = address(3);
    address public bob = address(4);
```

```

uint256 public constant FRACTION_SCALE = 1e18;
uint256 public constant DEFAULT_FRACTION = 1e17; // 1:10 ratio (10%)

event SingularityLocked(address indexed user, uint256 amount, uint256
totalLocked);
event SingularityUnlocked(address indexed user, uint256 amount, uint256
totalLocked);
event Deposited(address indexed user, uint256 usdtAmount, uint256 usddAmount,
uint256 sharesReceived, uint256 ethFee);
event WithdrawnToUSDT(address indexed user, uint256 sharesRedeemed, uint256
usddAmount, uint256 usdtReceived, uint256 ethFee);
event WithdrawnToUSDD(address indexed user, uint256 sharesRedeemed, uint256
usddReceived, uint256 ethFee);
event FractionUpdated(uint256 oldFraction, uint256 newFraction);
event MaxCapUpdated(uint256 oldCap, uint256 newCap);
event EmergencyWithdrawn(address indexed user, uint256 sharesRedeemed);

function setUp() public {
    // Deploy mock tokens
    singularityToken = new MockERC20("Singularity", "SLY", 18);
    usdt = new MockERC20("Tether USD", "USDT", 18);
    usdd = new MockERC20("Decentralized USD", "USDD", 18);

    // Deploy sUSDD vault
    sUsdd = new MockSUSDD(address(usdd));

    // Deploy PSM
    psm = new MockPSM(address(usdt), address(usdd));

    // Deploy fee manager
    feeManager = new MockSLYFeeManagement(feeReceiver);

    // Deploy USDDVault
    vm.prank(owner);
    vault = new USDDVault(
        address(singularityToken),
        address(usdt),
        address(usdd),
        address(sUsdd),
        address(psm),
        address(feeManager),
        owner,
        DEFAULT_FRACTION
    );

    // Setup: Mint tokens to users
    singularityToken.mint(alice, 1000e18);
    singularityToken.mint(bob, 1000e18);
    usdt.mint(alice, 100000e18);
    usdt.mint(bob, 100000e18);

```

```

// Fund PSM with tokens for swaps
usdd.mint(address(psm), 1000000e18);
usdt.mint(address(psm), 1000000e18);

// Fund sUSDD with USDD for redemptions
usdd.mint(address(sUsdd), 1000000e18);

// Give users some ETH for fees
vm.deal(alice, 10 ether);
vm.deal(bob, 10 ether);
}

// ===== Constructor Tests =====

function test_Constructor_Success() public {
    assertEq(address(vault.singularrayToken()), address(singularrayToken));
    assertEq(address(vault.usdt()), address(usdt));
    assertEq(address(vault.usdd()), address(usdd));
    assertEq(address(vault.sUsdd()), address(sUsdd));
    assertEq(address(vault.psm()), address(psm));
    assertEq(address(vault.slyFeeManagement()), address(feeManager));
    assertEq(vault.fraction(), DEFAULT_FRACTION);
    assertEq(vault.owner(), owner);
}

function test_Constructor_RevertInvalidAddresses() public {
    vm.startPrank(owner);

    vm.expectRevert(IUSDDVault.InvalidAddress.selector);
    new USDDVault(
        address(0), address(usdt), address(usdd), address(sUsdd),
        address(psm), address(feeManager), owner, DEFAULT_FRACTION
    );

    vm.expectRevert(IUSDDVault.InvalidAddress.selector);
    new USDDVault(
        address(singularrayToken), address(0), address(usdd), address(sUsdd),
        address(psm), address(feeManager), owner, DEFAULT_FRACTION
    );

    vm.expectRevert(IUSDDVault.InvalidFraction.selector);
    new USDDVault(
        address(singularrayToken), address(usdt), address(usdd),
address(sUsdd),
        address(psm), address(feeManager), owner, 0
    );

    vm.stopPrank();
}

// ===== Lock/Unlock Singularray Tests =====

```

```
function test_LockSingularity_Success() public {
    uint256 lockAmount = 100e18;

    vm.startPrank(alice);
    singularityToken.approve(address(vault), lockAmount);

    vm.expectEmit(true, false, false, true);
    emit SingularityLocked(alice, lockAmount, lockAmount);

    vault.lockSingularity(lockAmount);
    vm.stopPrank();

    IUSDDVault.UserPosition memory pos = vault.getUserPosition(alice);
    assertEq(pos.singularityLocked, lockAmount);
    assertEq(vault.totalSingularityLocked(), lockAmount);
    assertEq(singularityToken.balanceOf(address(vault)), lockAmount);
}

function test_LockSingularity_RevertZeroAmount() public {
    vm.prank(alice);
    vm.expectRevert(IUSDDVault.InvalidAmount.selector);
    vault.lockSingularity(0);
}

function test_LockSingularity_RevertWhenPaused() public {
    vm.prank(owner);
    vault.pause();

    vm.prank(alice);
    vm.expectRevert();
    vault.lockSingularity(100e18);
}

function test_UnlockSingularity_Success() public {
    uint256 lockAmount = 100e18;

    // Lock first
    vm.startPrank(alice);
    singularityToken.approve(address(vault), lockAmount);
    vault.lockSingularity(lockAmount);

    // Unlock
    vm.expectEmit(true, false, false, true);
    emit SingularityUnlocked(alice, lockAmount, 0);

    vault.unlockSingularity(lockAmount);
    vm.stopPrank();

    IUSDDVault.UserPosition memory pos = vault.getUserPosition(alice);
    assertEq(pos.singularityLocked, 0);
    assertEq(vault.totalSingularityLocked(), 0);
    assertEq(singularityToken.balanceOf(alice), 1000e18);
}
```

```
}

function test_UnlockSingularity_RevertInsufficientBalance() public {
    vm.prank(alice);
    vm.expectRevert(IUSDDVault.InsufficientBalance.selector);
    vault.unlockSingularity(100e18);
}

function test_UnlockSingularity_RevertCannotUnlockActiveCollateral() public {
    uint256 lockAmount = 200e18; // Lock 200 SLY (2x required)
    uint256 depositAmount = 1000e18; // Requires 100 SLY at 1:10 ratio

    // Lock and deposit
    vm.startPrank(alice);
    singularityToken.approve(address(vault), lockAmount);
    vault.lockSingularity(lockAmount);

    usdt.approve(address(vault), depositAmount);
    uint256 fee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: fee}(depositAmount);

    // Try to unlock all - should fail (need to keep 100 SLY)
    vm.expectRevert(IUSDDVault.CannotUnlockActiveCollateral.selector);
    vault.unlockSingularity(lockAmount);

    // Try to unlock too much - should fail
    vm.expectRevert(IUSDDVault.CannotUnlockActiveCollateral.selector);
    vault.unlockSingularity(150e18);

    // Can unlock excess (50 SLY, leaving 150 which is > 100 required)
    uint256 excess = 50e18;
    vault.unlockSingularity(excess);
    vm.stopPrank();

    IUSDDVault.UserPosition memory pos = vault.getUserPosition(alice);
    assertEq(pos.singularityLocked, lockAmount - excess);
}

// ===== Deposit Tests =====

function test_Deposit_Success() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

    vm.startPrank(alice);

    // Lock Singularity
    singularityToken.approve(address(vault), lockAmount);
    vault.lockSingularity(lockAmount);

    // Deposit USDT
    usdt.approve(address(vault), depositAmount);
```

```
uint256 fee = vault.calculateDepositFee(depositAmount);

vm.expectEmit(true, false, false, true);
emit Deposited(alice, depositAmount, depositAmount, depositAmount, fee);

vault.deposit{value: fee}(depositAmount);
vm.stopPrank();

IUSDDVault.UserPosition memory pos = vault.getUserPosition(alice);
assertEq(pos.usdtDeposited, depositAmount);
assertEq(pos.sUsddShares, depositAmount); // 1:1 initially
assertEq(vault.totalUsdtDeposited(), depositAmount);
assertEq(vault.totalSUsddShares(), depositAmount);
}

function test_Deposit_RevertInsufficientSingularrayLocked() public {
    uint256 lockAmount = 50e18; // Not enough for 1000 USDT
    uint256 depositAmount = 1000e18;

    vm.startPrank(alice);
    singularrayToken.approve(address(vault), lockAmount);
    vault.lockSingularray(lockAmount);

    usdt.approve(address(vault), depositAmount);
    uint256 fee = vault.calculateDepositFee(depositAmount);

    vm.expectRevert(IUSDDVault.InsufficientSingularrayLocked.selector);
    vault.deposit{value: fee}(depositAmount);
    vm.stopPrank();
}

function test_Deposit_RevertInsufficientFee() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

    vm.startPrank(alice);
    singularrayToken.approve(address(vault), lockAmount);
    vault.lockSingularray(lockAmount);

    usdt.approve(address(vault), depositAmount);
    uint256 fee = vault.calculateDepositFee(depositAmount);

    vm.expectRevert(IUSDDVault.InsufficientFee.selector);
    vault.deposit{value: fee - 1}(depositAmount);
    vm.stopPrank();
}

function test_Deposit_RefundExcessETH() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

    vm.startPrank(alice);
```

```

    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);

    usdt.approve(address(vault), depositAmount);
    uint256 fee = vault.calculateDepositFee(depositAmount);
    uint256 excessETH = 0.1 ether;

    uint256 balanceBefore = alice.balance;
    vault.deposit{value: fee + excessETH}(depositAmount);
    uint256 balanceAfter = alice.balance;

    assertEq(balanceBefore - balanceAfter, fee);
    vm.stopPrank();
}

function test_Deposit_RevertExceedsMaxCap() public {
    uint256 maxCap = 5000e18;
    uint256 depositAmount1 = 3000e18;
    uint256 depositAmount2 = 2500e18; // Total: 5500, exceeds 5000 cap
    uint256 totalDeposit = depositAmount1 + depositAmount2;
    uint256 lockAmount = (totalDeposit * DEFAULT_FRACTION) / FRACTION_SCALE;
    // 550 SLV for 5500 USDT

    vm.prank(owner);
    vault.setMaxCapPerWallet(maxCap);

    vm.startPrank(alice);
    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);

    // First deposit (within cap)
    usdt.approve(address(vault), depositAmount1);
    uint256 fee1 = vault.calculateDepositFee(depositAmount1);
    vault.deposit{value: fee1}(depositAmount1);

    // Second deposit exceeds cap (3000 + 2500 = 5500 > 5000)
    usdt.approve(address(vault), depositAmount2);
    uint256 fee2 = vault.calculateDepositFee(depositAmount2);

    vm.expectRevert(IUSDDVault.ExceedsMaxCap.selector);
    vault.deposit{value: fee2}(depositAmount2);
    vm.stopPrank();
}

// ===== WithdrawToUSDT Tests =====

function test_WithdrawToUSDT_Success() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

    // Setup: lock and deposit
    vm.startPrank(alice);

```



```

    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);

    usdt.approve(address(vault), depositAmount);
    uint256 depositFee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: depositFee}(depositAmount);

    // Withdraw
    IUSDDVault.UserPosition memory posBefore = vault.getUserPosition(alice);
    uint256 sharesToRedeem = posBefore.sUsddShares;
    uint256 withdrawFee = vault.calculateWithdrawFee(depositAmount, true);

    uint256 usdtBefore = usdt.balanceOf(alice);

    vm.expectEmit(true, false, false, true);
    emit WithdrawnToUSDT(alice, sharesToRedeem, depositAmount, depositAmount,
withdrawFee);

    vault.withdrawToUSDT{value: withdrawFee}(sharesToRedeem);
    vm.stopPrank();

    uint256 usdtAfter = usdt.balanceOf(alice);
    assertEq(usdtAfter - usdtBefore, depositAmount);

    IUSDDVault.UserPosition memory posAfter = vault.getUserPosition(alice);
    assertEq(posAfter.sUsddShares, 0);
    assertEq(posAfter.usdtDeposited, 0);
}

function test_WithdrawToUSDT_WithYield() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

    // Setup: lock and deposit
    vm.startPrank(alice);
    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);

    usdt.approve(address(vault), depositAmount);
    uint256 depositFee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: depositFee}(depositAmount);
    vm.stopPrank();

    // Simulate 10% yield
    sUsdd accrueYield(1000); // 10% = 1000 bps

    // Check accrued yield
    uint256 yield = vault.getAccruedYield(alice);
    assertGt(yield, 0);
    assertEq(yield, 100e18); // 10% of 1000

    // Withdraw

```

```

    vm.startPrank(alice);
    IUSDDVault.UserPosition memory pos = vault.getUserPosition(alice);
    uint256 currentValue = vault.getCurrentShareValue(alice);
    uint256 withdrawFee = vault.calculateWithdrawFee(currentValue, true);

    uint256 usdtBefore = usdt.balanceOf(alice);
    vault.withdrawToUSDT{value: withdrawFee}(pos.sUsddShares);
    uint256 usdtAfter = usdt.balanceOf(alice);

    assertEq(usdtAfter - usdtBefore, currentValue); // Should get 1100 USDT
    vm.stopPrank();
}

function test_WithdrawToUSDT_PartialWithdraw() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

    // Setup
    vm.startPrank(alice);
    singularrayToken.approve(address(vault), lockAmount);
    vault.lockSingularray(lockAmount);

    usdt.approve(address(vault), depositAmount);
    uint256 depositFee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: depositFee}(depositAmount);

    // Withdraw half
    IUSDDVault.UserPosition memory posBefore = vault.getUserPosition(alice);
    uint256 sharesToRedeem = posBefore.sUsddShares / 2;
    uint256 expectedUsdt = depositAmount / 2;
    uint256 withdrawFee = vault.calculateWithdrawFee(expectedUsdt, true);

    vault.withdrawToUSDT{value: withdrawFee}(sharesToRedeem);
    vm.stopPrank();

    IUSDDVault.UserPosition memory posAfter = vault.getUserPosition(alice);
    assertEq(posAfter.sUsddShares, posBefore.sUsddShares / 2);
    assertEq(posAfter.usdtDeposited, depositAmount / 2);
}

function test_WithdrawToUSDT_RevertInsufficientBalance() public {
    vm.prank(alice);
    vm.expectRevert(IUSDDVault.InsufficientBalance.selector);
    vault.withdrawToUSDT{value: 0.1 ether}(1000e18);
}

// ===== WithdrawToUSDD Tests =====

function test_WithdrawToUSDD_Success() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

```

```
// Setup
vm.startPrank(alice);
singularryToken.approve(address(vault), lockAmount);
vault.lockSingularry(lockAmount);

usdt.approve(address(vault), depositAmount);
uint256 depositFee = vault.calculateDepositFee(depositAmount);
vault.deposit{value: depositFee}(depositAmount);

// Withdraw to USDD
IUSDDVault.UserPosition memory pos = vault.getUserPosition(alice);
uint256 withdrawFee = vault.calculateWithdrawFee(depositAmount, false);

uint256 usddBefore = usdd.balanceOf(alice);

vm.expectEmit(true, false, false, true);
emit WithdrawnToUSDD(alice, pos.sUsddShares, depositAmount, withdrawFee);

vault.withdrawToUSDD{value: withdrawFee}(pos.sUsddShares);
vm.stopPrank();

uint256 usddAfter = usdd.balanceOf(alice);
assertEq(usddAfter - usddBefore, depositAmount);
}

function test_WithdrawToUSDD_WithYield() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

    // Setup
    vm.startPrank(alice);
    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);

    usdt.approve(address(vault), depositAmount);
    uint256 depositFee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: depositFee}(depositAmount);
    vm.stopPrank();

    // Simulate 20% yield
    sUsdd accrueYield(2000);

    // Withdraw
    vm.startPrank(alice);
    IUSDDVault.UserPosition memory pos = vault.getUserPosition(alice);
    uint256 currentValue = vault.getCurrentShareValue(alice);
    uint256 withdrawFee = vault.calculateWithdrawFee(currentValue, false);

    uint256 usddBefore = usdd.balanceOf(alice);
    vault.withdrawToUSDD{value: withdrawFee}(pos.sUsddShares);
    uint256 usddAfter = usdd.balanceOf(alice);
```

```
    assertEq(usddAfter - usddBefore, currentValue); // Should get 1200 USDD
    vm.stopPrank();
}

// ===== View Functions Tests =====

function test_GetUserPosition() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

    vm.startPrank(alice);
    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);

    usdt.approve(address(vault), depositAmount);
    uint256 fee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: fee}(depositAmount);
    vm.stopPrank();

    IUSDDVault.UserPosition memory pos = vault.getUserPosition(alice);
    assertEq(pos.singularryLocked, lockAmount);
    assertEq(pos.usdtDeposited, depositAmount);
    assertEq(pos.sUsddShares, depositAmount);
    assertGt(pos.depositTimestamp, 0);
}

function test_GetRequiredSingularry() public {
    uint256 depositAmount = 1000e18;

    vm.startPrank(alice);
    singularryToken.approve(address(vault), 200e18);
    vault.lockSingularry(200e18);

    usdt.approve(address(vault), depositAmount);
    uint256 fee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: fee}(depositAmount);
    vm.stopPrank();

    uint256 required = vault.getRequiredSingularry(alice);
    assertEq(required, 100e18); // 10% of 1000
}

function test_GetUnlockableSingularry() public {
    uint256 lockAmount = 200e18;
    uint256 depositAmount = 1000e18;

    vm.startPrank(alice);
    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);

    usdt.approve(address(vault), depositAmount);
    uint256 fee = vault.calculateDepositFee(depositAmount);
```

```
vault.deposit{value: fee}(depositAmount);
vm.stopPrank();

uint256 unlockable = vault.getUnlockableSingularry(alice);
assertEq(unlockable, 100e18); // Locked 200, required 100
}

function test_GetAccruedYield() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

    vm.startPrank(alice);
    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);

    usdt.approve(address(vault), depositAmount);
    uint256 fee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: fee}(depositAmount);
    vm.stopPrank();

    // No yield initially
    assertEq(vault.getAccruedYield(alice), 0);

    // Simulate 15% yield
    sUsdd accrueYield(1500);

    uint256 yield = vault.getAccruedYield(alice);
    assertEq(yield, 150e18); // 15% of 1000
}

function test_PreviewDeposit() public {
    uint256 depositAmount = 1000e18;
    uint256 expectedShares = vault.previewDeposit(depositAmount);
    assertEq(expectedShares, depositAmount); // 1:1 initially
}

function test_PreviewWithdraw() public {
    uint256 shares = 1000e18;
    (uint256 usddAmount, uint256 usdtAmount) = vault.previewWithdraw(shares);
    assertEq(usddAmount, shares);
    assertEq(usdtAmount, shares);
}

function test_GetExchangeRate() public {
    uint256 rate = vault.getExchangeRate();
    assertEq(rate, 1e18); // 1:1 initially

    // Simulate yield
    sUsdd accrueYield(500); // 5%
    rate = vault.getExchangeRate();
    assertEq(rate, 1.05e18);
}
```

```
// ===== Admin Functions Tests =====

function test_SetFraction_Success() public {
    uint256 newFraction = 2e17; // 20%

    vm.prank(owner);
    vm.expectEmit(false, false, false, true);
    emit FractionUpdated(DEFAULT_FRACTION, newFraction);

    vault.setFraction(newFraction);

    assertEq(vault.fraction(), newFraction);
}

function test_SetFraction_RevertZero() public {
    vm.prank(owner);
    vm.expectRevert(IUSDDVault.InvalidFraction.selector);
    vault.setFraction(0);
}

function test_SetFraction_RevertNotOwner() public {
    vm.prank(alice);
    vm.expectRevert();
    vault.setFraction(2e17);
}

function test_SetMaxCapPerWallet_Success() public {
    uint256 newCap = 10000e18;

    vm.prank(owner);
    vm.expectEmit(false, false, false, true);
    emit MaxCapUpdated(0, newCap);

    vault.setMaxCapPerWallet(newCap);

    assertEq(vault.maxCapPerWallet(), newCap);
}

function test_SetMaxCapPerWallet_RevertNotOwner() public {
    vm.prank(alice);
    vm.expectRevert();
    vault.setMaxCapPerWallet(10000e18);
}

function test_Pause_Success() public {
    vm.prank(owner);
    vault.pause();

    // Should revert when trying to lock
    vm.prank(alice);
    vm.expectRevert();
}
```

```
        vault.lockSingularry(100e18);
    }

    function test_Pause_RevertNotOwner() public {
        vm.prank(alice);
        vm.expectRevert();
        vault.pause();
    }

    function test_Unpause_Success() public {
        vm.startPrank(owner);
        vault.pause();
        vault.unpause();
        vm.stopPrank();

        // Should work after unpause
        vm.startPrank(alice);
        singularryToken.approve(address(vault), 100e18);
        vault.lockSingularry(100e18);
        vm.stopPrank();
    }

    // ===== Fee Tests =====

    function test_FeeCalculation() public {
        uint256 amount = 1000e18;
        uint256 fee = vault.calculateDepositFee(amount);
        assertGt(fee, 0);

        // Fee should be sent to fee receiver
        uint256 receiverBalanceBefore = feeReceiver.balance;

        vm.startPrank(alice);
        singularryToken.approve(address(vault), 100e18);
        vault.lockSingularry(100e18);

        usdt.approve(address(vault), amount);
        vault.deposit{value: fee}(amount);
        vm.stopPrank();

        uint256 receiverBalanceAfter = feeReceiver.balance;
        assertEq(receiverBalanceAfter - receiverBalanceBefore, fee);
    }

    // ===== Integration Tests =====

    function test_FullCycle_WithYield() public {
        uint256 lockAmount = 100e18;
        uint256 depositAmount = 1000e18;

        // Alice locks and deposits
        vm.startPrank(alice);
```

```

    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);

    usdt.approve(address(vault), depositAmount);
    uint256 depositFee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: depositFee}(depositAmount);
    vm.stopPrank();

    // Check initial state
    IUSDDVault.UserPosition memory pos1 = vault.getUserPosition(alice);
    assertEq(pos1.usdtDeposited, depositAmount);
    assertEq(pos1.sUsddShares, depositAmount);

    // Simulate 25% yield
    sUsdd accrueYield(2500);

    // Check yield
    uint256 yield = vault.getAccruedYield(alice);
    assertEq(yield, 250e18);

    // Withdraw
    vm.startPrank(alice);
    uint256 currentValue = vault.getCurrentShareValue(alice);
    assertEq(currentValue, 1250e18);

    uint256 withdrawFee = vault.calculateWithdrawFee(currentValue, true);
    uint256 usdtBefore = usdt.balanceOf(alice);

    vault.withdrawToUSDT{value: withdrawFee}(pos1.sUsddShares);
    vm.stopPrank();

    uint256 usdtAfter = usdt.balanceOf(alice);
    assertEq(usdtAfter - usdtBefore, 1250e18); // Principal + yield

    // Check final state
    IUSDDVault.UserPosition memory pos2 = vault.getUserPosition(alice);
    assertEq(pos2.sUsddShares, 0);
    assertEq(pos2.usdtDeposited, 0);

    // Can now unlock all Singularry
    vm.prank(alice);
    vault.unlockSingularry(lockAmount);

    assertEq(singularryToken.balanceOf(alice), 1000e18);
}

function test_MultiplUsers() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

    // Alice deposits
    vm.startPrank(alice);

```



```

    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);
    usdt.approve(address(vault), depositAmount);
    uint256 fee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: fee}(depositAmount);
    vm.stopPrank();

    // Bob deposits
    vm.startPrank(bob);
    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);
    usdt.approve(address(vault), depositAmount);
    vault.deposit{value: fee}(depositAmount);
    vm.stopPrank();

    // Check totals
    assertEq(vault.totalSingularryLocked(), lockAmount * 2);
    assertEq(vault.totalUsdtDeposited(), depositAmount * 2);
    assertEq(vault.totalUSddShares(), depositAmount * 2);

    // Simulate yield
    sUsdd accrueYield(1000); // 10%

    // Both should have yield
    assertEq(vault.getAccruedYield(alice), 100e18);
    assertEq(vault.getAccruedYield(bob), 100e18);

    // Alice withdraws
    vm.startPrank(alice);
    IUSDDVault.UserPosition memory posAlice = vault.getUserPosition(alice);
    uint256 withdrawFee = vault.calculateWithdrawFee(1100e18, true);
    vault.withdrawToUSDT{value: withdrawFee}(posAlice.sUsddShares);
    vm.stopPrank();

    // Bob's position should be unaffected
    assertEq(vault.getAccruedYield(bob), 100e18);
}

function test_EdgeCase_TinyDeposit() public {
    uint256 lockAmount = 1e18;
    uint256 depositAmount = 10e18; // Tiny deposit

    vm.startPrank(alice);
    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);

    usdt.approve(address(vault), depositAmount);
    uint256 fee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: fee}(depositAmount);
    vm.stopPrank();

    IUSDDVault.UserPosition memory pos = vault.getUserPosition(alice);

```

```
    assertEq(pos.usdtDeposited, depositAmount);
    assertGt(pos.sUsddShares, 0);
}

function test_EdgeCase_MaxDeposit() public {
    uint256 maxDeposit = 1000000e18;
    uint256 requiredSly = (maxDeposit * DEFAULT_FRACTION) / FRACTION_SCALE;

    // Mint enough tokens
    singularryToken.mint(alice, requiredSly);
    usdt.mint(alice, maxDeposit);

    vm.startPrank(alice);
    singularryToken.approve(address(vault), requiredSly);
    vault.lockSingularry(requiredSly);

    usdt.approve(address(vault), maxDeposit);
    uint256 fee = vault.calculateDepositFee(maxDeposit);
    vm.deal(alice, fee);
    vault.deposit{value: fee}(maxDeposit);
    vm.stopPrank();

    assertEq(vault.totalUsdtDeposited(), maxDeposit);
}

// ===== Emergency Withdrawal Tests =====

function test_EmergencyWithdrawShares_Success() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

    // Setup: Alice deposits
    vm.startPrank(alice);
    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);
    usdt.approve(address(vault), depositAmount);
    uint256 fee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: fee}(depositAmount);
    vm.stopPrank();

    // Simulate 10% yield
    sUsdd accrueYield(1000);

    // Owner pauses due to PSM failure
    vm.prank(owner);
    vault.pause();

    // Alice withdraws shares directly
    vm.startPrank(alice);
    IUSDDVault.UserPosition memory posBefore = vault.getUserPosition(alice);
    uint256 sharesToWithdraw = posBefore.sUsddShares;
```

```

uint256 sUsddBefore = IERC20(address(sUsdd)).balanceOf(alice);

vm.expectEmit(true, false, false, true);
emit EmergencyWithdrawn(alice, sharesToWithdraw);

vault.emergencyWithdrawShares();

uint256 sUsddAfter = IERC20(address(sUsdd)).balanceOf(alice);

assertEq(sUsddAfter - sUsddBefore, sharesToWithdraw);

IUSDDVault.UserPosition memory posAfter = vault.getUserPosition(alice);
assertEq(posAfter.sUsddShares, 0);
assertEq(posAfter.usdtDeposited, 0);
vm.stopPrank();

// Alice can now redeem sUSDD directly from sUSDD vault
vm.startPrank(alice);
IERC20(address(sUsdd)).approve(address(sUsdd), sharesToWithdraw);
uint256 usddReceived = sUsdd.redeem(sharesToWithdraw, alice, alice);
assertGt(usddReceived, 1000e18); // Should include yield (1100 USDD)
assertEq(usddReceived, 1100e18);
vm.stopPrank();
}

function test_EmergencyWithdrawShares_RevertWhenNotPaused() public {
    vm.prank(alice);
    vm.expectRevert(Pausable.ExpectedPause.selector);
    vault.emergencyWithdrawShares();
}

function test_EmergencyWithdrawShares_RevertZeroShares() public {
    vm.prank(owner);
    vault.pause();

    vm.prank(alice);
    vm.expectRevert(IUSDDVault.InvalidAmount.selector);
    vault.emergencyWithdrawShares();
}

function test_EmergencyWithdrawShares_WithYield() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

    // Setup
    vm.startPrank(alice);
    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);
    usdt.approve(address(vault), depositAmount);
    uint256 fee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: fee}(depositAmount);
    vm.stopPrank();
}

```

```
// Simulate 25% yield
sUsdd accrueYield(2500);

// Check yield before emergency withdrawal
uint256 yieldBefore = vault.getAccruedYield(alice);
assertEq(yieldBefore, 250e18);

// Owner pauses
vm.prank(owner);
vault.pause();

// Alice emergency withdraws
vm.startPrank(alice);
vault.emergencyWithdrawShares();
vm.stopPrank();

// Alice redeems sUSDD shares - should get principal + yield
vm.startPrank(alice);
uint256 aliceShares = IERC20(address(sUsdd)).balanceOf(alice);
IERC20(address(sUsdd)).approve(address(sUsdd), aliceShares);
uint256 usddReceived = sUsdd.redeem(aliceShares, alice, alice);
assertEq(usddReceived, 1250e18); // 1000 + 250 yield
vm.stopPrank();
}

function test_EmergencyWithdrawShares_MultipleUsers() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

    // Alice and Bob deposit
    vm.startPrank(alice);
    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);
    usdt.approve(address(vault), depositAmount);
    uint256 fee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: fee}(depositAmount);
    vm.stopPrank();

    vm.startPrank(bob);
    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);
    usdt.approve(address(vault), depositAmount);
    vault.deposit{value: fee}(depositAmount);
    vm.stopPrank();

    // Owner pauses
    vm.prank(owner);
    vault.pause();

    // Alice withdraws
    vm.prank(alice);
```

```

    vault.emergencyWithdrawShares();

    // Check Alice's position cleared
    IUSDDVault.UserPosition memory posAlice = vault.getUserPosition(alice);
    assertEq(posAlice.sUsddShares, 0);

    // Check Bob's position unaffected
    IUSDDVault.UserPosition memory posBob = vault.getUserPosition(bob);
    assertEq(posBob.sUsddShares, depositAmount);

    // Bob can also withdraw
    vm.prank(bob);
    vault.emergencyWithdrawShares();

    posBob = vault.getUserPosition(bob);
    assertEq(posBob.sUsddShares, 0);
}

function test_EmergencyWithdrawShares_ThenUnlockSingularry() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;

    // Setup
    vm.startPrank(alice);
    singularryToken.approve(address(vault), lockAmount);
    vault.lockSingularry(lockAmount);
    usdt.approve(address(vault), depositAmount);
    uint256 fee = vault.calculateDepositFee(depositAmount);
    vault.deposit{value: fee}(depositAmount);
    vm.stopPrank();

    // Pause and emergency withdraw
    vm.prank(owner);
    vault.pause();

    vm.prank(alice);
    vault.emergencyWithdrawShares();

    // Unpause
    vm.prank(owner);
    vault.unpause();

    // Now Alice can unlock all Singularry since deposit is cleared
    vm.prank(alice);
    vault.unlockSingularry(lockAmount);

    IUSDDVault.UserPosition memory pos = vault.getUserPosition(alice);
    assertEq(pos.singularryLocked, 0);
    assertEq(singularryToken.balanceOf(alice), 1000e18);
}
}

```

2. USDDVaultUnitTestOutput

```
Ran 45 tests for test/USDDVault.t.sol:USDDVaultTest
[PASS] test_Constructor_RevertInvalidAddresses() (gas: 289755)
[PASS] test_Constructor_Success() (gas: 34701)
[PASS] test_Deposit_RefundExcessETH() (gas: 443523)
[PASS] test_Deposit_RevertExceedsMaxCap() (gas: 496346)
[PASS] test_Deposit_RevertInsufficientFee() (gas: 175562)
[PASS] test_Deposit_RevertInsufficientSingularlyLocked() (gas: 171807)
[PASS] test_Deposit_Success() (gas: 444375)
[PASS] test_EdgeCase_MaxDeposit() (gas: 453099)
[PASS] test_EdgeCase_TinyDeposit() (gas: 439528)
[PASS] test_EmergencyWithdrawShares_MultipleUsers() (gas: 780261)
[PASS] test_EmergencyWithdrawShares_RevertWhenNotPaused() (gas: 18400)
[PASS] test_EmergencyWithdrawShares_RevertZeroShares() (gas: 50103)
[PASS] test_EmergencyWithdrawShares_Success() (gas: 554112)
[PASS] test_EmergencyWithdrawShares_ThenUnlockSingularly() (gas: 503731)
[PASS] test_EmergencyWithdrawShares_WithYield() (gas: 547723)
[PASS] test_FeeCalculation() (gas: 438886)
[PASS] test_FullCycle_WithYield() (gas: 579462)
[PASS] test_GetAccruedYield() (gas: 450078)
[PASS] test_GetExchangeRate() (gas: 23790)
[PASS] test_GetRequiredSingularly() (gas: 438826)
[PASS] test_GetUnlockableSingularly() (gas: 438969)
[PASS] test_GetUserPosition() (gas: 439616)
[PASS] test_LockSingularly_RevertWhenPaused() (gas: 46116)
[PASS] test_LockSingularly_RevertZeroAmount() (gas: 19030)
[PASS] test_LockSingularly_Success() (gas: 122281)
[PASS] test_MultiplUsers() (gas: 826443)
[PASS] test_Pause_RevertNotOwner() (gas: 13525)
[PASS] test_Pause_Success() (gas: 46159)
[PASS] test_PreviewDeposit() (gas: 12881)
[PASS] test_PreviewWithdraw() (gas: 13038)
[PASS] test_SetFraction_RevertNotOwner() (gas: 14035)
[PASS] test_SetFraction_RevertZero() (gas: 13994)
[PASS] test_SetFraction_Success() (gas: 23516)
[PASS] test_SetMaxCapPerWallet_RevertNotOwner() (gas: 14080)
[PASS] test_SetMaxCapPerWallet_Success() (gas: 40676)
[PASS] test_UnlockSingularly_RevertCannotUnlockActiveCollateral() (gas: 460936)
[PASS] test_UnlockSingularly_RevertInsufficientBalance() (gas: 21236)
[PASS] test_UnlockSingularly_Success() (gas: 122154)
[PASS] test_Unpause_Success() (gas: 123667)
[PASS] test_WithdrawToUSDD_Success() (gas: 520969)
[PASS] test_WithdrawToUSDD_WithYield() (gas: 529171)
[PASS] test_WithdrawToUSDT_PartialWithdraw() (gas: 550616)
[PASS] test_WithdrawToUSDT_RevertInsufficientBalance() (gas: 27940)
[PASS] test_WithdrawToUSDT_Success() (gas: 555520)
[PASS] test_WithdrawToUSDT_WithYield() (gas: 563892)
```

```
Suite result: ok. 45 passed; 0 failed; 0 skipped; finished in 2.99ms (10.94ms CPU time)
```

```
Ran 1 test suite in 139.99ms (2.99ms CPU time): 45 tests passed, 0 failed, 0 skipped (45 total tests)
```

3. USDDVaultFacet.t.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "forge-std/Test.sol";
import "../src/slywallet/facets/usdd/SLYWalletUSDDVaultFacet.sol";
import "../src/sly_usdd_vault/USDDVault.sol";
import "../src/sly_usdd_vault/interfaces/IUSDDVault.sol";
import "../src/common/mocks/MockERC20.sol";
import "../src/common/mocks/MockPSM.sol";
import "../src/common/mocks/MockSLYFeeManagement.sol";
import "../src/common/mocks/MockSUSDD.sol";

contract SLYWalletUSDDVaultFacetTest is Test {
    SLYWalletUSDDVaultFacet public wallet;
    USDDVault public vault;
    MockERC20 public singularrayToken;
    MockERC20 public usdt;
    MockERC20 public usdd;
    MockSUSDD public sUsdd;
    MockPSM public psm;
    MockSLYFeeManagement public feeManager;

    address public owner = address(1);
    address public feeReceiver = address(2);
    address public user = address(this);

    uint256 public constant FRACTION_SCALE = 1e18;
    uint256 public constant DEFAULT_FRACTION = 1e17; // 1:10 ratio (10%)

    function setUp() public {
        // Deploy mock tokens
        singularrayToken = new MockERC20("Singularray", "SLY", 18);
        usdt = new MockERC20("Tether USD", "USDT", 18);
        usdd = new MockERC20("Decentralized USD", "USDD", 18);

        // Deploy sUSDD vault
        sUsdd = new MockSUSDD(address(usdd));

        // Deploy PSM
        psm = new MockPSM(address(usdt), address(usdd));
    }
}
```

```

// Deploy fee manager
feeManager = new MockSLYFeeManagement(feeReceiver);

// Deploy USDDVault
vault = new USDDVault(
    address(singularrayToken),
    address(usdt),
    address(usdd),
    address(sUsdd),
    address(psm),
    address(feeManager),
    owner,
    DEFAULT_FRACTION
);

// Deploy wallet facet
wallet = new SLYWalletUSDDVaultFacet();

// Mint tokens to wallet
singularrayToken.mint(address(wallet), 1000e18);
usdt.mint(address(wallet), 100000e18);
usdd.mint(address(psm), 1000000e18);
usdt.mint(address(psm), 1000000e18);
usdd.mint(address(sUsdd), 1000000e18);
vm.deal(address(wallet), 10 ether);

// Initialize facet
wallet.initializeUSDDVault(address(vault));
}

function test_LockSingularray() public {
    uint256 lockAmount = 100e18;
    wallet.vaultLockSingularray(lockAmount);
    IUSDDVault.UserPosition memory pos =
vault.getUserPosition(address(wallet));
    assertEq(pos.singularrayLocked, lockAmount);
}

function test_UnlockSingularray() public {
    uint256 lockAmount = 100e18;
    wallet.vaultLockSingularray(lockAmount);
    wallet.vaultUnlockSingularray(lockAmount);
    IUSDDVault.UserPosition memory pos =
vault.getUserPosition(address(wallet));
    assertEq(pos.singularrayLocked, 0);
}

function test_DepositAndWithdraw() public {
    uint256 lockAmount = 200e18; // Lock more for two deposits
    uint256 depositAmount = 1000e18;
    wallet.vaultLockSingularray(lockAmount);

```



```
// Test 1: Deposit and withdraw to USDT
wallet.vaultDeposit(depositAmount);
IUSDDVault.UserPosition memory pos =
vault.getUserPosition(address(wallet));
assertEq(pos.usdtDeposited, depositAmount);
assertEq(pos.sUsddShares, depositAmount);

wallet.vaultWithdrawToUSDT(depositAmount);
pos = vault.getUserPosition(address(wallet));
assertEq(pos.sUsddShares, 0);
assertEq(pos.usdtDeposited, 0);

// Test 2: Deposit and withdraw to USDD
wallet.vaultDeposit(depositAmount);
pos = vault.getUserPosition(address(wallet));
assertEq(pos.usdtDeposited, depositAmount);
assertEq(pos.sUsddShares, depositAmount);

wallet.vaultWithdrawToUSDD(depositAmount);
pos = vault.getUserPosition(address(wallet));
assertEq(pos.sUsddShares, 0);
assertEq(pos.usdtDeposited, 0);
}

function test_EmergencyWithdrawShares() public {
    uint256 lockAmount = 100e18;
    uint256 depositAmount = 1000e18;
    wallet.vaultLockSingularly(lockAmount);
    wallet.vaultDeposit(depositAmount);
    // Pause vault
    vm.prank(owner);
    vault.pause();
    // Emergency withdraw
    wallet.vaultEmergencyWithdrawShares();
    IUSDDVault.UserPosition memory pos =
    vault.getUserPosition(address(wallet));
    assertEq(pos.sUsddShares, 0);
    assertEq(pos.usdtDeposited, 0);
}
}
```

4. USDDVaultFacetUnitTestOutput

```
Ran 4 tests for test/SLYWalletUSDDVaultFacet.t.sol:SLYWalletUSDDVaultFacetTest
[PASS] test_DepositAndWithdraw() (gas: 917269)
[PASS] test_EmergencyWithdrawShares() (gas: 513804)
[PASS] test_LockSingularity() (gas: 141904)
[PASS] test_UnlockSingularity() (gas: 139376)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 1.21ms (1.00ms CPU
time)

Ran 1 test suite in 123.21ms (1.21ms CPU time): 4 tests passed, 0 failed, 0
skipped (4 total tests)
```

11.2 External Functions Check Points

1. USDDVault.sol_output.md

File: src/sly_usdd_vault/USDDVault.sol

contract: USDDVault is IUSDDVault, Ownable2Step, ReentrancyGuard, Pausable

(Empty fields in the table represent things that are not required or relevant)

Index	Function	StateMutability	Modifier	Param Check	IsUserInterface	Unit Test	Miscellaneous
1	lockSingularry(uint256)		nonReentrant, whenNotPaused		Yes	Passed	
2	unlockSingularry(uint256)		nonReentrant, whenNotPaused		Yes	Passed	
3	deposit(uint256)	payable	nonReentrant, whenNotPaused		Yes	Passed	
4	withdrawToUSDT(uint256)	payable	nonReentrant, whenNotPaused		Yes	Passed	
5	withdrawToUSDD(uint256)	payable	nonReentrant, whenNotPaused		Yes	Passed	
6	getUserPosition(address)	view					
7	getAccruedYield(address)	view					
8	getCurrentShareValue(address)	view					
9	getRequiredSingularry(address)	view					
10	getUnlockableSingularry(address)	view					
11	previewDeposit(uint256)	view					
12	previewWithdraw(uint256)	view					
13	getExchangeRate()	view					
14	calculateDepositFee(uint256)	view					
15	calculateWithdrawFee(uint256,bool)	view					
16	emergencyWithdrawShares()		nonReentrant, whenPaused		Yes	Passed	
17	setFraction(uint256)		onlyOwner			Passed	
18	setMaxCapPerWallet(uint256)		onlyOwner			Passed	
19	pause()		onlyOwner			Passed	
20	unpause()		onlyOwner			Passed	

2. SLYWalletUSDDVaultFacet.sol_output.md

File: src/slywallet/facets/usdd/SLYWalletUSDDVaultFacet.sol

contract: SLYWalletUSDDVaultFacet is ISLYWalletUSDDVaultFacet, SLYWalletReentrancyGuard

(Empty fields in the table represent things that are not required or relevant)

Index	Function	StateMutability	Modifier	Param Check	IsUserInterface	Unit Test	Miscellaneous
1	initializeUSDDVault(address)		onlyAdmin			Passed	OnlyOnce
2	setUSDDVault(address)		onlyAdmin, onlyInitialized			Passed	
3	vaultLockSingularry(uint256)		onlyAdmin, onlyInitialized, nonReentrant			Passed	
4	vaultUnlockSingularry(uint256)		onlyAdmin, onlyInitialized, nonReentrant			Passed	
5	vaultDeposit(uint256)		onlyAdmin, onlyInitialized, nonReentrant			Passed	
6	vaultWithdrawToUSDT(uint256)		onlyAdmin, onlyInitialized, nonReentrant			Passed	
7	vaultWithdrawToUSDD(uint256)		onlyAdmin, onlyInitialized, nonReentrant			Passed	
8	vaultEmergencyWithdrawShares()		onlyAdmin, onlyInitialized, nonReentrant			Passed	
9	getUSDDVault()	view					
10	getVaultPosition()	view	onlyInitialized, returns, (IUSDDVault.UserPosition, memory)				
11	getVaultAccruedYield()	view	onlyInitialized				
12	getVaultShareValue()	view	onlyInitialized				
13	getVaultRequiredSingularry()	view	onlyInitialized				
14	getVaultUnlockableSingularry()	view	onlyInitialized				
15	getVaultDepositFee(uint256)	view	onlyInitialized				
16	getVaultWithdrawFee(uint256,bool)	view	onlyInitialized				
17	isUSDDVaultInitialized()	view					



<https://medium.com/@FairyproofT>



<https://twitter.com/FairyproofT>



<https://www.linkedin.com/company/fairyproof-tech>



https://t.me/Fairyproof_tech



Reddit: <https://www.reddit.com/user/FairyproofTech>

